Security Documentation

Dynamics Queue Admin — dispatch-queue management console (Single-Page Application), version 1.0.0. Security submission against the INSA Secure Website Management Standard, 2014 EC.

Document	**SEC-1.0.0** — Security Documentation
Asset name	Dynamics Queue Admin — queue organization / dispatch management console
Asset type	Web Application (client-side single-page application)
Asset version	1.0.0
Asset code	`ASSET-DYN-QA-001`
Repository	`queadmin-frontend`
Production host	`queue.dynamicsroute.tech`
Backend API	Separate system (`transportBackEndNative` , Node.js/Express + MySQL). Development API `https://dev.dynamicsroute.tech` ; production build defaults to the same-origin path `/api` unless `VITE_API_BASE_URL` is supplied at build time.
Governing standard	INSA Secure Website Management Standard, 2014 EC (`INSAREQUIREMENT.md`), Annexes A and B
Submitting entity	Dynamics Transport
Contact person	Marew Masresha, General Manager — `mmarew@gmail.com` , `0983222221`
Date of preparation	15 October 2026
Date of submission	15 October 2026

Honesty note on scope and status

This document covers **this repository and this asset only** — the Dynamics Queue Admin console SPA. The backend (`transportBackEndNative`) and the hosting platform are separate systems documented elsewhere; where a control depends on them this document says so rather than claiming it. Application-layer controls that are present in the submitted source are marked PASS. Server and hosting controls that are **absent entirely** (this repository contains no CSP, no security response headers and no nginx/server configuration), and organizational items with no evidence, are marked OPEN. Genuinely inapplicable tests are marked N/A. Ownership and contact fields are completed by the submitting entity.

Contents

1	Introduction
2	Asset and contact information
3	System description
4	Security requirements and objectives
5	Authentication and session management
6	Authorization and access control
7	Transport security
8	Application security controls
9	Input validation and data handling
10	Third-party and integration security
11	Dependency and supply-chain management
12	Deployment and hosting security
13	Logging, monitoring and incident response
14	Testing and assurance (Annex B checklist)
15	Annex A — Common vulnerabilities and threats
16	Compliance summary and deviations register
17	Security recommendations and roadmap
—	Document control, approvals and signatures

Related documents

Document	Identifier / location
Software Design Description	`SDD-1.0.0` — `docs/insa/SDD.html`
Technical Design / Low-Level Design	`TDD-1.0.0` — `docs/insa/TDD_LLD.html`
End User Guide	`docs/insa/UserGuide.html`
Standard and checklist	`INSAREQUIREMENT.md`

Document	Identifier / location
In-application security self-assessment	`src/pages/security/SecurityPage.tsx` with `src/data/insaStandard.ts` (5 focus areas, 35 requirements, 102 tests, 8 Annex A vulnerabilities)

How to read this document

Section 14 contains the full 102-item INSA Annex B minimum security testing checklist with a status for every item. Section 16 maps the five focus areas (2.1–2.5) to the organization's status. Section 17 gives the phased remediation roadmap. Where a control is described as implemented, the implementation point (file and symbol) is named so the reviewer can independently verify it. The source of truth for every negative statement is the submitted repository; no control is claimed that is not present in it.

1. Introduction

1.1 Purpose

This is the security documentation for **Dynamics Queue Admin** (`queadmin-frontend`), the dispatch-queue management console used by queue staff to manage driver queues, shipper requests, trips, queue organizations and real-time dispatch operations. It is submitted to the Information Network Security Agency (INSA) in support of the Secure Website Management Standard, 2014 EC. The document states the security requirements the asset is designed to meet, the controls actually implemented in the submitted release, the evidence for those controls, and — with equal prominence — the controls that are planned, outstanding, organizational, or owned by a separate system.

1.2 Scope

In scope: the client-side single-page application in this repository (React 19 + TypeScript + Vite 8), its production build, and its client-side security behaviours: session handling, route guards, form validation and sanitisation, storage handling, and error handling. The application is served at `queue.dynamicsroute.tech`.

Out of scope: the backend system of record (`transportBackEndNative`, Node.js/Express + MySQL), the driver mobile application, the shipper application, and the hosting platform itself. The backend is the enforcement boundary for authentication, authorization, session lifetime, upload handling and request validation; those obligations are identified here as dependencies and must be evidenced by the backend's own submission. The hosting platform (web server, TLS termination, certificate management, HTTP response headers) is a separate hosting engagement. **This repository contains no server configuration, no Content-Security-Policy and no security response headers; the document states this plainly rather than implying a server layer that does not exist in the source.**

1.3 Governing standard

The governing standard is the INSA *Secure Website Management Standard, 2014 EC*. It defines five focus areas — Requirement Gathering and Analysis (2.1), Design (2.2), Implementation (2.3), Hosting (2.4), and Operation and Management (2.5) — plus Annex A (eight common vulnerabilities and threats) and Annex B (a 102-item minimum security testing checklist). This document addresses all five focus areas and both annexes in turn.

1.4 Summary of outcome

The application layer is in reasonable shape. The console uses an httpOnly-cookie session model and never reads or stores a JWT in JavaScript (`getToken()` in `src/lib/auth.ts` deliberately returns `null`); login is one-time-password only, so there are no passwords to leak; forms are validated and sanitised with React Hook Form and Zod (`src/schemas/queue.ts` strips HTML tags and control characters); the client never renders untrusted HTML (a source scan found no `dangerouslySetInnerHTML`, `innerHTML`, `eval` or `new Function` in `src/`); logout clears the stored session profile and calls the backend logout; a global 401 handler ends the session on any unauthorized response; and production sourcemaps are disabled.

The largest gaps are real and are stated without mitigation. There is **no Content-Security-Policy and no security response header anywhere in the repository**, and no server configuration to deliver them, so anti-clickjacking, HSTS, anti-sniffing and related items cannot be evidenced from this asset.

The error utility `src/utils/parseError.ts` logs the **raw error object by default** (`console.error("[parseError]", error)`) and is not gated to development builds, so failure payloads can reach the browser console in production. The OTP is submitted to the backend as a JavaScript `Number`, which corrupts codes with a leading zero. The client also sends `roleId` and `statusId` on login and OTP verification even though the code comments and README claim it does not, so the backend must be confirmed to ignore them. There is no continuous integration, no dependency audit gate and no security linting plugin. Sections 14 and 16 quantify all of this precisely.

2. Asset and contact information

2.1 Asset detail

Attribute	Value
Asset name	Dynamics Queue Admin (dispatch-queue management console)
Repository / asset code	`queadmin-frontend` — asset code `ASSET-DYN-QA-001`
Asset type	Web Application — client-side single-page application
Version	1.0.0 (`package.json` private version is `0.0.0`)
Technology	React 19, TypeScript, Vite 8, react-router-dom 7, Redux Toolkit / RTK Query, Zustand, axios, socket.io-client 4, react-hook-form + zod, i18next, date-fns, sonner
Production URL	`https://queue.dynamicsroute.tech/`
API endpoint	Same-origin `/api` by default in production; development API `https://dev.dynamicsroute.tech` (separate backend system)
Real-time channel	Socket.IO with `withCredentials` to the backend (`VITE_WEBSOCKET_URL` / API base)
Distribution	Static bundle (HTML/JS/CSS) served from a document root (`dist/`)

2.2 Classification

Attribute	Value
Asset classification	Business / internal-use web application; commercially sensitive
Data sensitivity	Confidential operational data and personal data (driver and shipper names, phone numbers, email, vehicle and queue records)
Availability class	Business-operational (daily driver queue and dispatch operations)
Owner	Marew Masresha, General Manager (business owner); Marew Masresha, General Manager (system/technical owner)
Criticality	Business-critical to daily operations. Unavailability or compromise of the console disrupts loading-place driver queueing and dispatch at every organization using it, and can delay freight movements. Recoverability and integrity of queue and journey records are important; the records themselves are held by the backend system of record.

2.3 Ownership and points of contact

Role	Name	Contact
Business owner	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`

Role	Name	Contact
Technical / system owner	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`
Security contact	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`
Hosting provider contact	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`
Backend (API) team contact	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`
CERT / INSA reporting contact	Information Network Security Agency (INSA)	Via INSA reporting channel

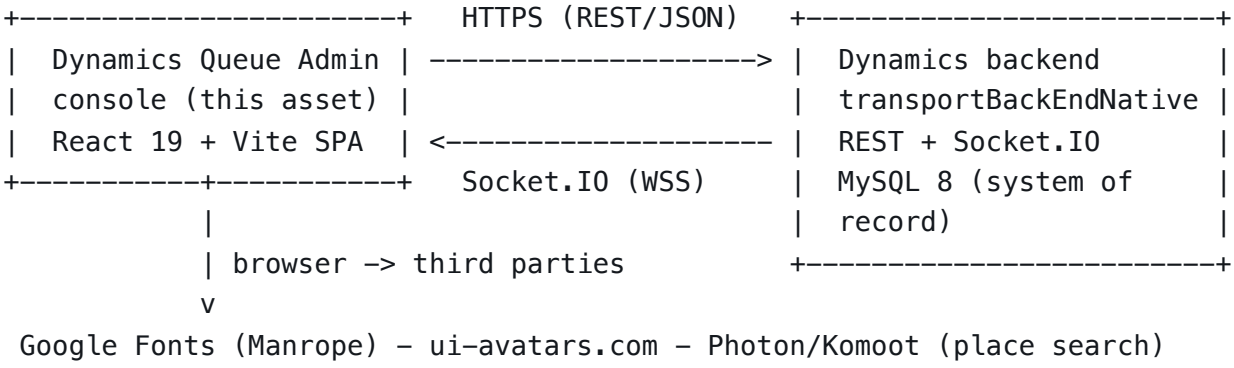
Contacts

Ownership and points of contact for Dynamics Transport are completed; the General Manager is the named business, technical, security and hosting contact. INSA reporting is made through the Agency's own channel, as shown above.

3. System description

3.1 Architecture

Dynamics Queue Admin is a standalone single-page application. It is loaded once from the static host, then communicates with a single backend over HTTPS (REST/JSON) and a single real-time channel (Socket.IO, with cookie credentials). It also calls third-party services directly from the browser for fonts, avatars and place search. The console is a queue-operations tool: queue organizations manage their driver queues, check drivers in, dispatch vehicles against shipper requests, and receive live queue updates. It holds no server-side logic and stores no database of its own; all state of record lives in the backend.



3.2 Security-relevant boundaries

Boundary	Description and control implication
Browser	The untrusted runtime. All client-side checks are conveniences, not security boundaries. There is no CSP in this asset, so the browser is not given a script-execution policy to enforce.
SPA client	Holds only a non-sensitive user profile in `localStorage`; the session JWT lives only in an httpOnly cookie set by the API host. As a cross-site (or same-origin, depending on configuration) cookie model, requests use `credentials: "include"`.
Backend API	The **authorization and validation enforcement boundary**. It derives identity and role from the session, enforces CSRF/session/rate-limit controls and re-validates all input. This asset cannot enforce these; see §6 and §16.
Socket (Socket.IO)	Live queue events. Uses `withCredentials: true` and negotiates `websocket` with a `polling` fallback (`src/lib/socket.ts`). Message authorization is a backend responsibility.
Third-party services	Google Fonts, ui-avatars.com and Photon/Komoot place search are contacted directly by the browser. These are points where data (including search text) can leave the Dynamics trust boundary; see §10.

3.3 Data-flow summary

Flow	Transport	Notes
Authentication	HTTPS + OTP	Phone number → OTP by SMS → verify (`POST /user/verifyUserByOTP`) → session cookie set by the backend; non-secret profile persisted locally. No password is used.
Session bootstrap	HTTPS, cookie	The profile is hydrated from `localStorage` on start for optimistic rendering; the backend is the authority, and any 401 on a subsequent request clears the session and returns the user to login.
Queue operations	HTTPS + cookie	REST/JSON for queue status, check-in, dispatch, shipper requests and organization management; authorized by the backend.
Real-time events	Socket.IO	Live queue and organization updates; RTK Query cache invalidation on receipt.
Place search	HTTPS to third party	Address/place text is sent to Photon/Komoot when a queue organization address is entered and not matched against the local Ethiopian-city list. §10.

4. Security requirements and objectives

The standard's objectives are expressed through the CI4A+Accountability model (Confidentiality, Integrity, Authentication, Authorization, Availability, and Accountability). The table below states the objective for this asset, the assets and data protected, and how the objective is met (and where it depends on the backend or hosting platform).

Objective	Assets / data protected	How it is met
Confidentiality	Queue operations, driver and shipper personal data, session cookie	TLS in production; httpOnly session cookie (no JS-readable token); logout clears the local profile; no secrets in the bundle; production sourcemaps disabled. Third-party place-search egress and raw error logging remain open (§9, §10).
Integrity	Queue entries, dispatch actions, shipper requests, organization records	Server-side validation and business rules authorize and persist all changes; the client presents data and forwards intent only. Client-side Zod schemas sanitise input before it is sent.
Authentication	User accounts (phone identity)	Phone number plus one-time password delivered by SMS. Single factor, accepted for this population; server-side OTP rate limiting is a required backend control (§5).
Authorization	Queue-organization-scoped operations and administration	Role-based access enforced by the backend; a client route-boundary guard (`RoleGuard`) prevents unauthorized UI from mounting. The backend is the enforcement boundary (§6).
Availability	Queue and dispatch services during operating hours	Static, cacheable bundle; the socket client reconnects with bounded attempts. The organization route check fails open on a transient error so a network blip does not lock operators out (a deliberate availability decision, noted in §6 and §16).
Accountability	Security events, changes, approvals	Application logging is development-oriented and, in one respect, unsafe in production (`parseError` logs raw errors by default). Server-side and organizational audit logging, change control, monitoring and risk assessment are outstanding (§13, §16).

> **Boundary discipline**
>
> Several objectives above are only partially satisfiable from this repository. The document names the owning system rather than implying this asset controls them. In particular, authentication strength, session lifetime, rate limiting, CSRF defence, cookie flags and per-request authorization are all backend obligations.

5. Authentication and session management

5.1 Authentication flow

1. The user enters a phone number; the console requests an OTP (`requestLoginOtp` , `POST /user/loginUser`).
2. A one-time password is delivered by SMS; the console collects six digits and verifies it (`verifyOtp` , `POST /user/verifyUserByOTP`).
3. On success the backend issues a session JWT in an **httpOnly cookie**. The console does not read or store the token: the RTK Query `onQueryStarted` handler stores only `userData` (`src/lib/redux/api/authEndpoints.ts`).
4. The console persists only the non-sensitive profile in `localStorage` under `queueadmin:user` so the UI can render the user's name and role across reloads (`src/lib/auth.ts`).

> **Implemented application-layer controls (verified in source)**
>
> - **No JS-readable token.** `getToken()` returns `null` by design; the lexically stored record is `{ userData }` only. Saved under `queueadmin:user` ; the legacy `queueadmin:auth` record that could have held a token is migrated and then deleted (`purgeLegacyAuth()`).
> - **Cookie-credential requests.** Both the axios layer (`src/lib/api.ts` , `withCredentials: true`) and the RTK Query base (`src/lib/redux/api/base.ts` , `credentials: "include"`) send the session cookie. No `Authorization` header is constructed.
> - **Logout clears state.** `AuthContext.logout()` disconnects the socket, calls `POST /user/logout` (best-effort, to clear the httpOnly cookie server-side), clears the stored profile via `clearAuth()` , and dispatches the redux `logout` action — Annex B #39, #63.
> - **Central 401 handling.** Any 401 on any API call ends the session: the axios interceptor clears auth and redirects to `/login` , and `baseQueryWithReauth` dispatches `logout()` . A 401 on the socket handshake also ends the session so the client does not reconnect-loop against a dead cookie (`src/lib/socket.ts`).
> - **Phone masking.** The OTP screen shows a masked number (`maskedPhone` in `src/hooks/useOtpVerification.ts`).

Findings that prevent Pass status (verified in source)

- **OTP transmitted as a number.** `verifyOtp` sends `OTP: Number(OTP)` (`src/services/auth.service.ts`). A six-digit code with a leading zero is transmitted with the zero stripped, so a valid code can be rejected (and the behaviour differs from the string-safe handling expected). Annex B #26 is marked **OPEN** on this ground.
- **roleId/statusId sent despite contrary comments.** Both `requestLoginOtp` and `verifyOtp` send `roleId: QUEUE_ORG_ADMIN_ROLE` (11) and `statusId: 1`, while the in-file comment and the README state that the client does *not* send a role. The backend must be confirmed to derive the role from the session and ignore these client-supplied values. Annex B #28, #36.
- **Raw error logging.** `parseError` logs the entire error object unless `silent` is passed, and it is not gated to development builds (`src/utils/parseError.ts:116`). This is discussed in §9 and is the basis for marking the token/log-disclosure items **OPEN**.
- **Server messages surfaced.** `resolveServerMsg` returns the backend's `message` / `error` text when present, so a failed OTP can display backend prose that may distinguish "unknown user" from "wrong code" (username enumeration). Annex B #35 is **OPEN**.

5.2 Session lifetime

Session lifetime is **backend-controlled and not visible to the client**. `VITE_JWT_EXPIRATION_DAYS=7` exists in `.env.development` / `.env.example` but is read nowhere in `src/`; it is misleading and should be removed. Per the backend session policy documentation (README, referencing `Utils/SessionPolicy.js`), sessions are role-dependent: **Admin (3) and SuperAdmin (6) = 24 hours; Queue Org Admin (11) and Queue Dispatcher (12) = 365 days.** The 365-day lifetime for queue staff is unusually long and is called out as a hardening recommendation (§17). A periodic-expiration requirement (Annex B #76) therefore rests entirely on the backend.

5.3 Residual items to confirm with the backend team

These cannot be implemented in this repository

Item	Why it matters
Cookie flags: `HttpOnly; Secure; SameSite`; `Path=/`	The production build defaults to a same-origin `/api` path, but a cross-site deployment (separate API host) would require `SameSite=None; Secure`. **Confirm the backend's actual configuration.** Annex B #67–#70.
Server-side OTP rate limiting and lockout	The console has only a 60-second resend cooldown (`RESEND_COOLDOWN_SECONDS = 60`). Without a server-side counter, OTPs can be brute-forced and SMS-toll abuse is possible. Annex B #37.
Server-side session revocation and regeneration	Regeneration on login and revocation on logout/credential change must happen server-side. Annex B #64–#66, #73.
Periodic session expiration and idle timeout	A maximum session age and idle timeout are backend decisions; the 365-day window should be reviewed. Annex B #76.
Session identifier quality	Unpredictability, collision resistance and transmission only over TLS are backend properties. Annex B #71, #72, #74, #75.
JWT returned in the response body	`VerifyOtpResponse` types a `token: string` field (`src/types/queue.ts`). If the cookie model is the target, the body token should be dropped or shortened. Annex B #36, #74.

6. Authorization and access control

6.1 Role model

The console is intended for queue-console roles: Queue Org Admin (role 11), Queue Dispatcher (role 12), platform Admin (role 3) and Super Admin (role 6). The role is part of the stored profile and is resolved by the backend at login. Membership in a specific queue organization is a backend relationship; the client cannot grant it.

6.2 Client route-boundary guards

Two components wrap protected routes and run *before* the child renders:

- `src/components/auth/RoleGuard.tsx` — the user must hold one of the allowed role IDs (`{ 11, 12, 3, 6 }`), otherwise the guard redirects to `/login`. Roles 3 and 6 are included because they are legitimate platform operators of the queue console.
- `src/components/auth/ProtectedRoute.tsx` — the user must be authenticated and must have at least one queue organization, otherwise they are redirected to `/setup-org`. It waits for the organization query to settle before deciding.

> **Fail-open behaviour in ProtectedRoute (Annex B #32)**
>
> When the organization query returns an error, `ProtectedRoute` returns `true` for `hasOrg` and allows the user through: `orgsError ? true : false` (`src/components/auth/ProtectedRoute.tsx:48–52`). This is a deliberate availability choice — a transient backend failure should not lock operators out of a live queue — but it is a fail-open condition and is marked **OPEN** under Annex B #32. It does not bypass authentication (the session check precedes it) nor backend authorization; it only affects the client's organization-setup gate. The recommended remediation is to distinguish an authorization failure (redirect) from a transport failure (show a retryable error, not the app).

> **The backend is the enforcement boundary — explicit statement**
>
> The route guards are a **UI safety control, not an authorization boundary**. A user who forges their stored profile or calls the API directly must still be denied by the backend. The backend must:
>
> - derive `roleId` / `statusId` from the session, never from the request body (the client sends them for legacy reasons — see §5.1);
> - scope every queue-organization query and mutation to the cookie identity;
> - authorize every organization and queue identifier against the session's memberships.
>
> Until the backend team evidences these, Annex B #23, #24, #27 and #28 cannot be closed.

6.3 Per-action checks

Individual actions present only the affordances the role permits (for example, bid approval is shown only to role 11 in the bidding board), and every mutating request is sent to the backend, which is solely responsible for accepting or rejecting it. No client-side decision is treated as authoritative.

6.4 Privilege categories and lifecycle

The code implements privilege categorization (INSA 2.5.D) through the role set above and the queue-organization membership model. Granting and revoking access, and rotating credentials on a fixed schedule, are organizational procedures that are not evidenced in this repository (INSA 2.5.C and 2.5.E).

7. Transport security

7.1 HTTPS

The production URL is HTTPS-only (`queue.dynamicsroute.tech`). The application is configured to call the API over `https://` in development (`VITE_API_BASE_URL=https://dev.dynamicsroute.tech`) and defaults to the same-origin `/api` path in a production build, which inherits the page's TLS. The socket client targets the same host and sends the cookie with `withCredentials: true`.

> **Not independently verified for this submission**
>
> This repository contains no server configuration, so the HTTP→HTTPS redirect, the TLS protocol/cipher set, and the certificate chain on the production host were **not independently assessed** here. The application itself only ever addresses HTTPS origins in production. Annex B #13 and #15 are marked PASS on the basis of the application's own transport choices (HTTPS/WSS and no mixed content); #14 is marked PASS with the platform redirect noted as unverified; #16 (HSTS) and #17–#18 (TLS/certificate) are OPEN.

7.2 Socket transport

`src/lib/socket.ts` initializes Socket.IO with `transports: ["websocket", "polling"]`, `withCredentials: true`, `reconnectionAttempts: 5`, `reconnectionDelay: 1500`, `reconnectionDelayMax: 5000` and `timeout: 20000`. The polling fallback is retained for restrictive networks; unlike a purely WebSocket-pinned client, the polling handshake is an HTTP request that must also be protected by the platform's cookie and CSRF controls.

7.3 HSTS and cookie scope

No `Strict-Transport-Security` header is defined anywhere in the repository, so HSTS is **absent** (Annex B #16). Because a production build defaults to a same-origin API path, the session cookie may be same-site; if the deployment instead splits the SPA and API across registrable domains, the cookie must be `SameSite=None; Secure` and CSRF defence cannot rely on the default `SameSite=Lax` (see §5.3 and §14 #61, #62). This ambiguity must be resolved by the deployment and the backend before submission.

8. Application security controls

8.1 Content-Security-Policy

> **There is no CSP — stated plainly**
>
> A full-repository scan (`src/`, `public/`, `index.html`, `vite.config.ts`) found **no Content-Security-Policy** — no meta tag, no build-time injection, and no server configuration to deliver a header. Unlike a hardened SPA, this asset provides no script-execution policy to the browser. Annex B #83 (non-specific content types), #85 (anti-sniffing) and #96 (anti-clickjacking) are therefore **OPEN**, and there is no defence-in-depth layer behind React's output encoding for Annex B #43.

Note that `index.html` embeds an inline `<script type="application/ld+json">` structured-data block. A strict `script-src` policy introduced later would need to accommodate it with a hash or nonce, or the block should be moved to an external file. This is recorded as a dependency of the remediation in §17.

8.2 Security response headers

No security response headers are configured in this repository: `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, `Cross-Origin-Opener-Policy` and `Cross-Origin-Resource-Policy` are all absent. The hosting platform may add some of these, but that is outside this asset and is **not evidenced**. They are recorded as hosting items in §16.

8.3 Client-side output encoding

- **React escaping.** All untrusted values are rendered through JSX text nodes and attribute bindings, which React escapes by default (Annex B #43).
- **No raw HTML sinks.** A source scan found no `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function` or `document.write` in `src/`.
- **Input sanitisation.** Zod transforms strip HTML tags and control characters from names, addresses and emails before they are stored or sent (`src/schemas/queue.ts`: `stripTags`, `stripCtrl`, `sanitizeText`, `sanitizeEmail`).

Without a CSP, these controls are the only layer preventing script injection; the recommendation in §17 is to add a policy as the first server-side change.

8.4 Build hardening

- **Sourcemap suppression.** `vite.config.ts` sets `build.sourcemap = process.env.NODE_ENV !== "production"`, so development builds carry sourcemaps and production builds do not. This is effective but implicit; pinning an explicit `false` for production is recommended (Annex B #6).
- **No secrets in the bundle.** Only public configuration (API/socket hosts) is referenced from environment variables; no API keys or credentials are embedded or committed.

9. Input validation and data handling

9.1 Form validation and sanitisers

Forms use `react-hook-form` with `zod` schemas (`src/schemas/queue.ts`): login, registration, organization setup, check-in and dispatch. Schemas constrain length and format, and transform values to strip HTML tags (`stripTags`), remove non-printable control characters (`stripCtrl`), collapse whitespace (`sanitizeText`) and normalise email (`sanitizeEmail`). Phone numbers are normalised to E.164 (`+251…`) by `src/utils/phoneFormatter.ts` and `src/lib/authConfig.ts`. Client validation is a usability and hygiene control only; the backend must re-validate every field.

9.2 File uploads

> **No file upload feature**
>
> The console does not upload or download files: a source scan found no `FormData`, file input, `FileReader` or multipart request. The Annex B File Upload group (#78–#82) is therefore marked **N/A** for this asset. If uploads are added later (for example, compliance documents), the validation and storage controls must be designed and added to this document.

9.3 Production error handling and log hygiene

> **Finding: raw error logging is not dev-gated**
>
> `src/utils/parseError.ts` runs `console.error("[parseError]", error)` unless the caller passes `{ silent: true }`. It is **not** gated behind `import.meta.env.DEV`. The logged error object can include the request configuration and response data (which for auth calls can carry the phone number and OTP). This is a production information-disclosure and log-hygiene gap, and it is the reason Annex B #4 (sensitive information in logs/errors) and #77 (disclosure of tokens in logs) are marked **OPEN**, and it also contributes to #7.

The user-facing path is better: `parseError` returns mapped, generic messages for HTTP statuses (401 → "Session expired", 403 → "You do not have permission", etc.). However, when the backend supplies a `message` / `error` string it is surfaced to the user (`resolveServerMsg`), which can disclose internal detail. Consider returning only server messages from an allow-list.

9.4 Client-side data and PII handling

The session JWT lives only in the httpOnly cookie; `localStorage` holds the non-secret profile (`queueadmin:user`: name, phone, email, role) used for rendering. On logout, `clearAuth()` removes that profile from both `localStorage` and `sessionStorage`. Other local keys (`app_language`, `queueadmin:store` — a UI selection only, `insa_security_checklist_v1`, theme and sidebar preferences) hold no personal data and are not cleared; this is acceptable but is noted for completeness. The residual risk is that the profile is stored in cleartext while signed in; this is a deliberate routing decision and is recorded in §16.

10. Third-party and integration security

The console contacts third parties directly from the browser. Each is listed with its actual call site; no other external origin is referenced in the source.

Third party	Purpose	Security notes
Google Fonts (`fonts.googleapis.com`)	Manrope typeface	Loaded via an `@import` at the top of `src/index.css`. Static assets only; no application data sent, but it is an external dependency on the critical rendering path and a request to a third party on every page load.
ui-avatars.com	Generated avatar images	Used in `DashboardLayout.tsx:42`, `DesktopProfileSection.tsx:44` and `MobileProfileView.tsx:41`. The user's display name is placed in the query string (`?name=…`), so a portion of personal data is disclosed to the third party and appears in its access logs. Self-hosting avatar generation would remove this egress.
Photon / Komoot (`photon.komoot.io`)	Address / place search autocomplete	**PII egress.** `src/hooks/usePhotonSearch.ts` sends the user's typed place/address text to Photon when it is not matched against the local Ethiopian-city list, debounced at 350 ms. Queue-organization addresses and search terms therefore leave the Dynamics boundary. Configurable via `VITE_PHOTON_URL`; a backend proxy or a self-hosted instance is the recommended remediation (§17).

> **No CSP restricts these origins**
>
> Because the asset has no Content-Security-Policy, the browser is not constrained from contacting any origin. Introducing a policy that allow-lists only these three origins (and the backend) would both document and mechanically enforce the third-party surface. Until then, Annex B #8 and #11 remain **OPEN**.

11. Dependency and supply-chain management

11.1 Dependency auditing

`npm audit --omit=dev` reports **one high-severity vulnerable runtime dependency**. The remediation is a one-command fix (`npm audit fix`) but is **not yet applied** and is not enforced anywhere. This fails Annex B #90 (missing security updates).

Package	Range	Severity	Notable advisories
`axios`	1.0.0 – 1.19.0	High	Prototype-pollution gadgets (fetch adapter, default instance, toFormData options), ReDoS in the `data:` URL parser, redirect-based SSRF, HTTP/2 adapter DNS/proxy bypass, and header injection via inherited headers (12+ advisories)

11.2 Static analysis and security linting

The ESLint configuration (`eslint.config.mjs`) uses `@eslint/js`, `typescript-eslint`, `eslint-plugin-react` and `eslint-plugin-react-hooks` only. There is **no security plugin** (no `eslint-plugin-security`), so risky patterns such as non-literal regular expressions, timing comparisons or unsafe object injection are not flagged. `oxlint` is installed and configured but is not a security gate. The `lint` script is `eslint . --ext ts,tsx --max-warnings 0`, while the README describes `npm run lint` as oxlint — the documentation and the script disagree.

11.3 A gap: unused and mis-declared dependencies

- `@tanstack/react-query` is listed in `package.json` but **nothing imports it**; the project uses RTK Query. It is dead weight and expands the attack surface and audit scope for no benefit.
- `.npmrc` sets `legacy-peer-deps=true`, which suppresses peer dependency resolution and weakens reproducibility of the install graph.

11.4 Continuous integration

> **No CI today**
>
> There is no `.github/` directory, no Dependabot configuration, no secret scanning and no build pipeline. Nothing catches dependency drift, ensures a clean rebuild on deploy, or runs the test suite (107 tests, all passing) automatically. A recommended workflow is given in §17.

12. Deployment and hosting security

12.1 Deployment model

The console is built to a static bundle (`npm run build`: `tsc -b && vite build`) and served from a document root (`dist/`). The production host serves a static site and terminates TLS. The development server proxies `/api` and `/socket.io` to the backend (`vite.config.ts`).

12.2 Server configuration status

This repository contains no server configuration at all — no nginx file, no header set, no 404 blocks, no caching policy. The table below therefore states the hosting-layer items as unverified rather than claiming a proposed configuration. They must be completed and evidenced by the hosting engagement.

Item	Status	Notes
Security response headers	ABSENT / UNVERIFIED	No header set exists in the repository; platform behaviour unknown.
Content-Security-Policy	ABSENT	No meta tag, no build injection, no header.
Extraneous-file 404 blocks	UNVERIFIED	No `/.env`, `/.git`, `*.map` guard is defined.
Cache-Control policy	UNVERIFIED	HTML should be `no-store`; hashed assets `immutable`.
HTTP→HTTPS redirect	UNVERIFIED	Assumed at the platform; not evidenced here.
TLS termination	UNVERIFIED	Platform manages certificates; configuration not independently assessed.
Staging environment	ABSENT	Patch/config changes cannot be tested on a separate environment (INSA 2.5.M).

12.3 Organizational hosting items

The standard's hosting focus area (2.4) is largely organizational and cannot be satisfied by repository artifacts: a hosting security requirements document, assessment of the provider's technical and non-technical capability, hosting location, and a comprehensive security agreement. These are recorded as outstanding in §16.

13. Logging, monitoring and incident response

13.1 Current state

- Application logging is development-oriented but **not safe in production** in one respect: `parseError` logs the raw error object by default and is not dev-gated. The socket client also logs connection errors via `console.warn` with the underlying error object (`src/lib/socket.ts`).
- There is no client-side transmission of security events, no server-side audit log evidenced for this asset, and no alerting.
- Because there is no CSP, there is no violation-reporting capability (`report-uri` / `report-to`).

13.2 Gaps

Gap	INSA reference
No breach or misuse monitoring for the web application	2.5.K (current status: Fail)
No access/audit logging of security-relevant events	2.5.B, 2.5.K
No periodic security testing or auditing programme	2.5.G, 2.5.H
No documented incident response or breach-notification procedure	2.4.E.ii, 2.5.A.ii

13.3 Recommendations

- Gate `parseError`'s raw logging behind `import.meta.env.DEV` and log only a terse, payload-free object in production (immediate, code-only).
- Enable structured application monitoring and attack/misuse detection for the public host.
- Configure a CSP report endpoint once a policy is deployed and triage violations.
- Define a documented breach-handling and emergency-communication procedure with named owners.
- Establish the CERT/INSA reporting contact and the escalation path — reporting is made to the Information Network Security Agency (INSA) through its published channel, with the General Manager as the named internal escalation point.

14. Testing and assurance

14.1 Automated tests

The repository has a Vitest suite that is **independently verifiable**: the current run reports **107 passing tests across 7 files** (`npm test`, executed during the preparation of this document). The suite includes security-relevant coverage:

Test file	Tests	Coverage
`src/__tests__/insaStandard.test.ts`	6	Structural integrity of the embedded INSA model: 5 focus areas, 35 requirements, 102 Annex B tests, 8 Annex A vulnerabilities.
`src/__tests__/securityChecklist.test.ts`	14	Whitelisting/sanitisation of the checklist state stored in localStorage: invalid, oversized and wrong-typed entries are dropped.
`src/__tests__/queueMutations.test.ts`	40	Queue business logic and mutation validation, including `parseError` behaviour and status-transition rules.
`src/__tests__/journeyStatus.test.ts`	19	Journey/order status mapping.
`src/__tests__/phoneFormatter.test.ts`	12	Phone normalisation and masking behaviour.
`src/__tests__/ordersLocation.test.ts`	12	Location parsing and validation.
`src/__tests__/ordersDataMapper.test.ts`	4	Order payload mapping.

The suite is **not run in CI** because there is no CI. There are no automated tests for the auth flow, route guards or socket handling.

> **How to read the Annex B table**
>
> PASS = implemented and verified in the submitted source (a specific file is named). OPEN = failing, absent, dependent on the separate backend, or organizational with no evidence. N/A = genuinely not applicable to this asset (for example, a static SPA has no server-side XML parser and executes no uploaded file; this console performs no file uploads). Server and hosting rows are left OPEN because **no server configuration exists in the repository** to verify them against.

14.2 Annex B — minimum security testing checklist (102 items)

No	Test	Status
Information Disclosure		
1	Test for extraneous files in the document root — no server configuration or 404 guard for `/.env`, `/.git`, `*.map`; live host behaviour not evidenced	OPEN
2	Test for extraneous directory listings — no server configuration to confirm autoindex is disabled	OPEN

No	Test	Status
3	Test for accessible debug functionality — no debug pages in the application, but server-side debug endpoints are not evidenced	OPEN
4	Test for sensitive information in log and error messages — `parseError` logs the raw error object by default and is not dev-gated	OPEN
5	Test for sensitive information in `robots.txt` — contains only `Allow: /` and the sitemap URL; nothing sensitive	PASS
6	Test for sensitive information in source code — no secrets in the bundle; production sourcemaps disabled	PASS
7	Test for disclosure of internal addresses — no internal addresses in the bundle; residual disclosure through surfacing of raw server messages is noted in §9	PASS
Privacy and Confidentiality		
8	Test for sensitive information stored in URLs — place-search text is sent to Photon/Komoot; no `Referrer-Policy` exists; avatar names are placed in ui-avatars URLs	OPEN
9	Test for unencrypted sensitive information stored at the client-side — logout purges the stored profile (name, phone, email) from both storages; remaining local keys hold no personal data	PASS
10	Test for sensitive information stored in (externally) archived pages — no `Cache-Control` policy defined or verified	OPEN
11	Test for content included from untrusted sources — Google Fonts, ui-avatars.com, Photon/Komoot; third-party data egress with no CSP to constrain it	OPEN
12	Test for caching of pages with sensitive information — no `Cache-Control` configuration exists in the repository	OPEN
13	Test for insecure transmission of sensitive information — production URL and API are HTTPS; socket uses WSS/HTTPS with cookie credentials	PASS
14	Test for non-SSL/TLS pages on sites processing sensitive information — the application only ever addresses HTTPS origins in production; host redirect not independently evidenced	PASS
15	Test for SSL/TLS pages served with mixed content — all fetched resources (fonts, avatars, API, socket) are HTTPS; no mixed content	PASS
16	Test for missing HSTS header on full SSL sites — no HSTS is defined anywhere in the repository	OPEN
17	Test for known vulnerabilities in SSL/TLS — platform TLS configuration not independently assessed	OPEN
18	Test for weak, untrusted or expired SSL certificates — platform certificate not independently assessed	OPEN
19	Test for the usage of unproven cryptographic primitives — the SPA performs no custom cryptography; relies on browser TLS	N/A

No	Test	Status
20	Test for the incorrect usage of cryptographic primitives — no custom cryptographic implementation in the asset	N/A
State Management		
21	Test for client-side state management — non-secret profile held in `localStorage` plus smaller UI keys; not encrypted while signed in; mitigated by server-side authority and logout purge	OPEN
22	Test for invalid state transitions — business state machine enforced by the backend, not this asset	OPEN
Authentication and Authorization Process		
23	Test for missing authentication or authorization — route-boundary guards exist; backend must enforce per request; `ProtectedRoute` includes one fail-open branch	OPEN
24	Test for client-side authentication — `isAuthenticated` /role derived from the locally stored profile; the backend is the boundary	OPEN
25	Test for predictable and default credentials — OTP-only, so no default passwords; backend assurance of OTP strength and delivery required	OPEN
26	Test for predictable authentication or authorization tokens — OTP is cast to `Number` before transmission, corrupting codes with a leading zero (`src/services/auth.service.ts`)	OPEN
27	Test for authentication or authorization based on obscurity — role/membership enforced by the backend; client guard is UI only	OPEN
28	Test for identifier-based authorization / privilege escalation — the client sends `roleId` (11) and `statusId` (1) on login/OTP despite comments claiming otherwise; every organization/queue identifier must be authorized server-side	OPEN
29	Test for acceptance of weak passwords — no password authentication is used (phone + OTP)	N/A
30	Test for account recovery process — account recovery, if any, is backend-owned	OPEN
31	Test any “remember me” function — no such function exists in the console	N/A
32	Test for fail-open conditions — `ProtectedRoute` allows the user through when the organization query errors	OPEN
33	Test any impersonation function / reCAPTCHA — no impersonation function; CAPTCHA is a backend decision	OPEN
34	Test for plaintext retrieval of passwords — no passwords are stored or retrieved	N/A
35	Test for username enumeration / username uniqueness — raw backend `message` / `error` text is surfaced to the user when present (`resolveServerMsg`), allowing distinct OTP/unknown-user responses	OPEN
36	Check for unsafe distribution of credentials — `VerifyOtpResponse` types a `token` field returned in the response body; backend should review	OPEN

No	Test	Status
37	Test for missing rate limiting on authentication functionality — the client has only a 60 s resend cooldown; server-side limiting required	OPEN
38	Test for missing re-authentication when changing credentials — backend-owned	OPEN
39	Test for missing logout functionality — logout disconnects the socket, calls the backend logout, clears both storages and dispatches the redux logout	PASS
40	Test any multi-stage mechanisms — multi-stage flows are backend-owned	OPEN
User Input Management		
41	Test for SQL injection — no SQL in this asset; backend responsibility	OPEN
42	Test for path traversal and filename injection — the console has no file upload and accepts no filesystem path input, so there is no traversal surface	N/A
43	Test for cross-site scripting (stored, DOM and reflected) — React output encoding; source scan found no `dangerouslySetInnerHTML` / `innerHTML` ; Zod sanitisers strip tags and control characters. No CSP defence-in-depth exists (noted)	PASS
44	Test for system command injection — no command execution in a browser SPA	N/A
45	Test for XML injection — no XML parsing in this asset	N/A
46	Test for XPath injection — no XPath processing	N/A
47	Test for XSL(T) injection — no XSLT processing	N/A
48	Test for SMTP injection — no mail sending from this asset	N/A
49	Test for SSI injection — no server-side includes in this asset	N/A
50	Test for HTTP header injection — response headers are server-controlled; backend responsibility	OPEN
51	Test for HTTP parameter injection — client passes parameters to the backend; the server must validate	OPEN
52	Test for native software flaws (buffer overflow, integer bugs, format strings) — managed JS runtime; no native code in this asset	N/A
53	Test for LDAP injection — no LDAP processing	N/A
54	Test for dynamic scripting injection — source scan found no `eval` , `new Function` or `document.write` , so no dynamic-script sink exists; note this is enforced by review, not by a CSP	PASS
55	Test for regular expression injection — no security lint gate flags non-literal regex constructions; not assessed exhaustively	OPEN
56	Test for data property / field injection — wide request typings invite mass assignment; a server-side allow-list is required	OPEN
57	Test for protocol-specific injection — no custom protocol handling	N/A

No	Test	Status
58	Test for expression language injection — no expression language in this asset	N/A
59	Fuzz all request parameters — no fuzzing performed; backend owns request parsing	OPEN
60	Test for arbitrary redirection — the only programmatic redirect target is the constant `/login`; the only dynamic `href` is a `tel:` URI; no user-controlled redirect exists	PASS
Session Management		
61	Test for cross-site request forgery (CSRF) — cookie-based session; backend CSRF defence not evidenced	OPEN
62	Test for predictable CSRF tokens — no CSRF token in the client; backend responsibility	OPEN
63	Test for missing session revocation on logout — `clearAuth()` plus best-effort server logout and full storage purge; server revocation must be confirmed	PASS
64	Test for missing session regeneration on login — backend-owned	OPEN
65	Test for missing session regeneration when changing credentials — backend-owned	OPEN
66	Test for missing revocation of other sessions when changing credentials — backend-owned	OPEN
67	Test for missing `Secure` flag on session cookies — backend configuration unverified	OPEN
68	Test for missing `HttpOnly` flag on session cookies — intended httpOnly; backend configuration unverified; the client never reads a token	OPEN
69	Test for non-restrictive domain on session cookies — same-origin default versus cross-site deployment ambiguity; backend-owned	OPEN
70	Test for non-restrictive or missing path on session cookies — backend-owned	OPEN
71	Test for predictable session identifiers — backend-owned	OPEN
72	Test for session identifier collisions — backend-owned	OPEN
73	Test for session fixation — backend-owned	OPEN
74	Test for insecure transmission of session identifiers — cookie rides TLS only, but the JWT is also returned in the response body	OPEN
75	Test for external session hijacking — backend/transport property; cookie flags to be confirmed	OPEN
76	Test for missing periodic expiration of sessions — client env variable is dead; lifetimes are backend-only and are 24 h (roles 3/6) and 365 d (roles 11/12)	OPEN
77	Check for disclosure of tokens in logs — `parseError` logs the raw error object by default; request/response payloads for auth calls can carry the phone number and OTP	OPEN
File Upload		
78	Test for storage of uploaded files in the document root — no upload feature exists in this asset	N/A

No	Test	Status
79	Test for execution or interpretation of uploaded files — the console executes no uploaded file	N/A
80	Test for uploading outside of designated upload directory — no upload feature exists	N/A
81	Test for missing size restrictions on uploaded files — no upload feature exists	N/A
82	Test for missing type validation on uploaded files — no upload feature exists	N/A
Additional Checks		
83	Test for missing or non-specific content type definitions — no server/content-type configuration exists	OPEN
84	Test for missing character set definitions — `index.html` declares `<meta charset="UTF-8">`	PASS
85	Test for missing anti content sniffing measures — no `X-Content-Type-Options: nosniff` anywhere	OPEN
86	Test for XML external entity expansion — no XML processing in this asset	N/A
87	Test for external DTD parsing — no DTD processing	N/A
88	Test for extraneous or dangerous XML extensions — no XML subsystem	N/A
89	Test for recursive entity expansion — no XML entity processing	N/A
90	Test for missing security updates — `npm audit --omit=dev` reports a high-severity advisory for `axios` ; not yet fixed; no audit gate	OPEN
91	Test for unsupported or end-of-life software versions — no automated dependency gate exists	OPEN
92	Test for HTTP TRACK and TRACE methods / dangerous methods — no server configuration restricts methods; live behaviour not evidenced	OPEN
93	Test for extraneous functionality — no debug endpoints or dead application surfaces identified in `src/` ; server identity disclosure unverified	OPEN
94	Test for default credentials — OTP-only; default-credential management is backend/hosting	OPEN
95	Test for bugs in web server software — web server version not assessed	OPEN
96	Test for missing anti-clickjacking measures — no `frame-ancestors` and no `X-Frame-Options` ; a meta tag cannot deliver either	OPEN
97	Test for open redirection — no user-controlled redirect exists (only the constant `/login` target and `tel:` links)	PASS
98	Test for insecure cross-domain access policy — no `Permissions-Policy` /COOP/CORP headers exist	OPEN
99	Test for missing rate limiting on e-mail functionality — e-mail is backend-owned; none sent by this asset	OPEN

No	Test	Status
100	Test for missing rate limiting on resource-intensive functionality — backend-owned	OPEN
101	Test for inappropriate rate limiting resulting in a denial of service — backend dimensioning; not assessed	OPEN
102	Test for application- or setup-specific problems — no app-specific defects identified in this code review; continuous testing recommended	OPEN

14.3 Checklist result summary

Pass	Open	N/A	Total
14	63	25	102

The fourteen passing items are verified in the submitted source: #5, #6, #7, #9, #13, #14, #15, #39, #43, #54, #60, #63, #84 and #97. A substantial share of the 63 open items are backend, hosting or organizational obligations that cannot be closed from this repository. The client-side gaps that *can* be closed here are #4/#77 (dev-gate raw error logging), #26 (send the OTP as a string), #28/#35/#36 (stop sending roleId/statusId; stop surfacing raw server messages), and #55 (add a security lint gate); plus #90 (apply `npm audit fix`).

15. Annex A — Common vulnerabilities and threats

Annex A lists eight common website vulnerabilities and threats that must be considered during design, implementation, hosting, operation and management. The table states how the console is affected and the control applied (and to whom the control belongs).

Vulnerability / threat	Console exposure	Control applied
1. SQL Injection	The SPA issues no SQL; data access is entirely behind the backend API.	Backend responsibility (parameterised queries). Client sends only structured JSON over TLS. Marked **OPEN** pending backend evidence.
2. Unchecked Path Parameter / Directory Traversal	No filesystem path input, no file upload, and no server-supplied document opening in this console.	No traversal surface in this asset. Server document serving and upload directory confinement are backend/hosting responsibilities. **N/A** for the console.
3. Improper Session Management	Session cookie; optimistic local profile; role-dependent lifetime.	Implemented client-side: httpOnly-cookie model (token never JS-readable), central logout, full storage purge on logout, 401-driven session end, socket 401 handling. Backend must confirm cookie flags, periodic expiry, regeneration and revocation; the 365-day lifetime for roles 11/12 should be reviewed. Partially **OPEN**.
4. Cross-Site Scripting	Any untrusted string rendered into the DOM; server-supplied messages; OTP errors.	Implemented: React's output encoding, no raw HTML sinks (`dangerouslySetInnerHTML` / `innerHTML` absent), Zod sanitisers on form input. **No CSP exists**, so there is no script-execution policy as defence-in-depth. Marked **PASS** at the application layer with the CSP gap recorded in §17.
5. CSRF	Cookie-session authenticated state-changing REST calls.	Not controllable from this asset. Backend must implement CSRF defence (token or Origin checking) and set cookie `SameSite` correctly for its deployment topology. Marked **OPEN**.
6. HTTP Header Injection	Response headers and any reflected input reaching them.	No response headers are constructed by the SPA. Backend/hosting responsibility; no server configuration exists in the repository. Marked **OPEN**.
7. Mail Header Injection	The console sends no e-mail. OTP SMS delivery is backend-owned.	Not applicable to this asset. Backend responsibility for any mail/SMS composition. **N/A** for the console; backend assurance required.
8. Lack of Authentication and Authorization	Route-level UI access and every API call.	Implemented client-side: OTP authentication, `RoleGuard` role boundary, `ProtectedRoute` organization gate (with one fail-open branch). The **backend is the enforcement boundary** and must derive identity/role from the session and authorize every resource. Partially **OPEN** pending backend evidence.

16. Compliance summary and deviations register

16.1 Focus-area status

The status below reflects the evidence in this repository and the controls verifiable from it. It is deliberately blunt: application-layer controls that are implemented are distinguished from organizational, hosting and backend items that are outstanding or unknown. The five focus areas correspond to the standard's sections 2.1–2.5 and are also modelled inside the application itself in `src/data/insaStandard.ts` (35 requirements).

2.1 Requirement Gathering and Analysis

Req	Status	Notes
A–F	PARTIAL	Business requirements, security objectives and interfaces are implemented and documented in the SDD/TDD artefacts and the embedded standard model. Formal stakeholder sign-off evidence is not present.
G	UNKNOWN	Developer contracts addressing source-code intellectual property are not in evidence — organizational.
H	UNKNOWN	Top-management approval of the security requirements is not documented.

2.2 Design

Req	Status	Notes
A	PARTIAL	Security is designed into the asset (httpOnly-cookie session, role guards, Zod sanitisers, storage discipline) and documented in the SDD/TDD artefacts, but the design lacks a CSP/header strategy and no mapping to the Secure Software Development and Management Standard V.1 is evidenced.
B	UNKNOWN	Design review and approval by the responsible body is not documented.

2.3 Implementation

Req	Status	Notes
A	PARTIAL	Secure software standard not mapped; CI and a security lint baseline are outstanding.
B	UNKNOWN	No evidence of developer security training.
C	UNKNOWN	CMCSRS accreditation before hosting — organizational.
D	UNKNOWN	Pre-hosting audit — organizational.
E	PARTIAL	This repository contains no secrets; repository access control is not verified.

2.4 Hosting

Req	Status	Notes
A–C	UNKNOWN	No hosting security requirements document, and no assessment of the provider's technical or non-technical capability.
D	UNKNOWN	Hosting location unverified (relevant if the website is a government asset).
E	UNKNOWN	No comprehensive security agreement; the twelve named provisions are outstanding.

2.5 Operation and Management

Req	Status	Notes
A	UNKNOWN	No website management policy (administrative responsibilities, emergency communications, content management).
B–D	PARTIAL	`RoleGuard` implements privilege categorization (D) in code; documented operating procedures are unknown.
E	PARTIAL	Session revokes on logout; credential rotation interval unknown.
F	FAIL	No quarterly vulnerability and risk assessment evidence.
G–H	UNKNOWN	No periodic testing/auditing evidence.
I	PARTIAL	No CI or formal change-control procedure; changes are committed but not approved by a security owner.
J	UNKNOWN	No in-advance operational risk assessment for significant changes.
K	FAIL	No breach or misuse monitoring.
L–M	PARTIAL	Dependency remediation (`axios`) pending; no documented patch plan and no staging environment.
N	PARTIAL	The application uses HTTPS in production; certificate renewal and HSTS are not evidenced.
O	UNKNOWN	No monitoring of the provider's security updates.

> **Assessment**
>
> The application-layer controls are in reasonable shape. The compliance scaffolding is largely absent and the server layer provides no security headers at all. The largest gaps are the CSP/header layer (hosting), the raw error logging, the OTP/role findings, and continuous integration.

16.2 Deviations register

ID	Deviation	Justification / owner
D1	Backend controls (CSRF, cookie flags, OTP rate limiting, session regeneration and revocation, per-request authorization, derivation of role from the session) are not evidenced.	Owned by the separate `transportBackEndNative` system; to be covered by the backend submission. The client's sending of `roleId` / `statusId` makes this confirmation urgent. Risk accepted pending that evidence.
D2	No Content-Security-Policy and no security response headers (HSTS, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, COOP, CORP) exist in the repository.	Requires a hosting/server layer that is not part of this asset. Accepted as a documented deviation, not an unknown gap. Consequence: #16, #83, #85, #96, #98 remain open. Owner: platform/hosting team.
D3	No CI, dependency gate, or security lint baseline.	Remediation defined (§17). Risk accepted in the interim; the `axios` high-severity advisory is not yet fixed.
D4	Third-party PII egress via Photon place search and ui-avatars.	Functional requirement for address autocomplete and avatars. Mitigation is a backend proxy or local-only data; full fix in §17.
D5	Organizational items (2.1 G–H; 2.2 B; 2.3 B–D; all of 2.4; 2.5 A, F, K, O and others) have no evidence.	Outside engineering control; requires business action. Recorded as open, not claimed.
D6	Client profile (name, phone, email, role — non-secret) held in cleartext `localStorage` while signed in.	Deliberate routing decision; the profile carries no token and is cleared on logout. The backend remains the authority.
D7	Single-factor authentication (phone + OTP); no second factor or device binding.	Accepted for the queue-console population on connectivity grounds. Server-side OTP rate limiting is a precondition (§17).
D8	`ProtectedRoute` fails open when the organization query errors, and the 365-day session lifetime for roles 11/12.	Fail-open is an availability decision for live queue operations; the long lifetime is backend policy. Both to be reviewed, with the risk recorded in §14 (#32, #76).

17. Security recommendations and roadmap

The remediation is phased by risk reduction per unit effort. Phases 1–4 are code-only and independent of the platform decision; Phase 0 and Phase 6 require hosting or backend action and are the highest-impact changes. The recommended order is Phase 1 → Phase 2 → Phase 3 → Phase 4, with Phase 0 started immediately in parallel.

Phase 0 — Server/header layer (highest impact; needs hosting access)

Deliver a CSP and security-header layer on `queue.dynamicsroute.tech`. The policy must account for the inline JSON-LD block in `index.html` (hash or nonce, or move the block to an external file). Recommended header set: `Content-Security-Policy` (with `frame-ancestors 'none'`), `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy`, COOP/CORP, a `Cache-Control` policy (`no-store` on HTML, `immutable` on hashed assets), method restriction, and 404 blocks for `/.env`, `/.git` and `*.map`.

Phase	Scope	Blocks
0.1 Headers	CSP, HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, COOP, CORP	#16, #83, #85, #96, #98
0.2 Files & methods	404 blocks for `/.env`, `/.git`, `*.map`; method restriction; server identity suppression	#1, #2, #3, #92, #93
0.3 Caching	`no-store` on HTML, `immutable` on hashed assets	#12

Phase 1 — Stop the bleeding (half day, code-only)

#	Change	Annex B	Status
1.1	Gate `parseError` raw logging behind `import.meta.env.DEV`; log a terse, payload-free object in production	#4, #77	OPEN
1.2	Send the OTP as a string (`OTP: OTP`, not `Number(OTP)`)	#26	OPEN
1.3	Stop sending `roleId`/`statusId` on login and OTP, or reconcile the comments/README with reality	#28, #36	OPEN
1.4	Return only allow-listed server messages (no raw `message`/`error` passthrough) to close enumeration	#35	OPEN
1.5	`npm audit fix` and remove the unused `@tanstack/react-query`	#90	OPEN
1.6	Remove the dead `VITE_JWT_EXPIRATION_DAYS` variable	#76	OPEN

Phase 2 — Client hardening (1 day, code-only)

#	Change	Annex B	Status
2.1	Pin `build.sourcemap: false` explicitly for production	#6	OPEN
2.2	Change `ProtectedRoute` fail-open to distinguish transport failure (retryable message) from authorization failure (redirect)	#32	OPEN

#	Change	Annex B	Status
2.3	Add security linting (`eslint-plugin-security` or equivalent) with `detect-non-literal-regexp` enforced	#55	OPEN
2.4	Add tests for auth flow, route guards, 401 handling and socket unauthorized handling	#102	OPEN

Phase 3 — Third-party privacy (half day)

Self-host the Manrope font and avatar generation, and proxy Photon place search through the backend (or restrict it to the local Ethiopian-city list). This removes ui-avatars and Photon egress and keeps third-party data flows minimal.

Phase 4 — CI, so this cannot rot again (half day)

Add a CI workflow: `npm ci` → `npm run lint` → `npm run build` → `npm test` → `npm audit --audit-level=high`. Add Dependabot for `npm`, secret scanning, and gate on a clean lint baseline. Align the README's description of `npm run lint` with the actual ESLint script.

Phase 5 — Server policy source of truth (hosting)

Once a header layer exists, keep a single source of truth for the CSP (for example, a `src/constants/csp.ts` consumed by both build and server configuration) and add a drift guard so the two copies cannot diverge, plus a post-build assertion that the meta tag (or header) is present.

Phase 6 — Backend dependencies (separate repository)

Item	Annex B	Why
Confirm CSRF defence (double-submit or Origin checking)	#61, #62	Cookie-session authenticated mutations
Derive `roleId` / `statusId` from the session, never the body	#28	The client sends them for legacy reasons
Scope every queue/organization query to the cookie identity	#28	Ignore any client-supplied identifiers
Cookie flags: `HttpOnly; Secure`; correct `SameSite` for the deployment topology	#67–#70	Same-origin vs cross-site ambiguity must be resolved
Stop returning the JWT in the response body	#74	Target the cookie-only model
Server-side rate limiting on OTP and lockout	#37	Only a 60 s client cooldown exists
Server-side session regeneration, periodic expiry and revocation	#64–#66, #73, #76	365-day sessions and no regeneration are high-risk

Bottom line

Subject to the outstanding server, backend and organizational items above, the Dynamics Queue Admin console is suitable for continued operation and for INSA verification testing. The application-layer improvements listed as PASS are real and independently verifiable in the source. This document does not claim the production header layer is live, because no server configuration exists in the repository to evidence it.

Document control, approvals and signatures

Document control

Item	Value
Document title	Dynamics Queue Admin — Security Documentation
Document identifier	`SEC-1.0.0`
Version	1.0.0 (initial submission)
Asset	Dynamics Queue Admin web console (asset code `ASSET-DYN-QA-001`)
Governing standard	INSA Secure Website Management Standard, 2014 EC
Status	For INSA review
Classification	Confidential — submitted to INSA

Revision history

Version	Date	Author	Summary
0.1	15 October 2026	Marew Masresha, General Manager	Initial draft security documentation.
1.0.0	15 October 2026	Marew Masresha, General Manager	First issue for INSA submission.

Approval

Role	Name	Signature	Date
Prepared by	Marew Masresha, General Manager	_____	15 October 2026
Reviewed by	Marew Masresha, General Manager	_____	15 October 2026
Approved by	Marew Masresha, General Manager	_____	15 October 2026

Prepared by — Marew Masresha, General Manager
Signature — _____ Date — 15 October 2026

Reviewed by — Marew Masresha, General Manager
Signature — _____ Date — 15 October 2026

Approved by — Marew Masresha, General Manager
Signature — _____ Date — 15 October 2026